

llm-cost-router: a cost vs quality benchmark for cascade and learned LLM routers

Akshitha Reddy Lingampally

2024-08-15

Contents

1	Abstract	2
2	1. Background	2
2.1	1.1 Motivation	2
2.2	1.2 Scope	2
3	2. Related Work	2
4	3. Method	3
4.1	3.1 Cascade router	3
4.2	3.2 Learned router	3
4.3	3.3 Provider profiles	3
5	4. Data	3
6	5. Evaluation Setup	3
7	6. Results	4
7.1	6.1 Headline	4
7.2	6.2 Pareto	4
7.3	6.3 Per-router accuracy	4
7.4	6.4 Provider usage	4
7.5	6.5 Cost distribution	6
7.6	6.6 Accuracy by difficulty	6
8	7. Ablations	6
8.1	7.1 Cascade threshold	6
8.2	7.2 Learned-router features	6
9	8. Discussion	6
10	9. Limitations	7
11	10. Future Work	7

12 11. References	7
13 Appendix A. Reproducibility Checklist	7
14 Appendix B. Glossary	8

1 Abstract

llm-cost-router benchmarks three routing strategies (always-cheap, cascade, learned) across a synthetic difficulty-mixed query stream. On the bundled fixture (300 queries, 70/30 test/calibration split), the cascade router improves accuracy from 80% to 88% over the always-cheap baseline at 20x the total spend; the learned logistic-regression router with three features collapses to the cheap provider, showing that the chosen feature set is too thin to discriminate difficulty. The report explains the failure honestly and proposes the feature additions that would move the learned router off the cheap corner.

2 1. Background

2.1 1.1 Motivation

LLM serving costs are dominated by a small fraction of “hard” queries that need the expensive model. The opportunity is to send every other query to a cheaper provider without losing quality. Cascade routers do this with a confidence threshold; learned routers do this with a classifier. This project benchmarks both against an always-cheap baseline so the operator can see the cost vs quality Pareto directly.

2.2 1.2 Scope

- Three provider profiles (cheap, mid, expensive) with per-difficulty accuracies and per-call USD costs.
- A synthetic 300-query stream with controlled difficulty mix (50% easy, 30% medium, 20% hard).
- A cascade router that tries cheap first, escalates if confidence is low.
- A learned router (logistic regression on 3 features: prompt length, has_code, has_math) trained on a 30% calibration split.
- Five chart families.

3 2. Related Work

- **FrugalGPT** [Chen et al. 2023] popularized the cascade pattern.
- **RouteLLM** [Ong et al. 2024] used a learned router trained on preference data.
- **Mixture-of-LLMs** [Wang et al. 2024] generalized routing to ensembles.

4 3. Method

4.1 3.1 Cascade router

flowchart LR

```
A[Query] --> B[Cheap provider]
B --> C{Confidence >= threshold?}
C -- yes --> Z[Commit cheap answer]
C -- no --> D[Mid provider]
D --> E{Confidence >= threshold?}
E -- yes --> Z2[Commit mid answer]
E -- no --> F[Expensive provider]
F --> Z3[Commit expensive answer]
```

Threshold is a hyperparameter; the bundled value is 0.75.

4.2 3.2 Learned router

A three-feature logistic regression is trained on the calibration split. Labels are the index of the cheapest provider that gets the query right (simulated via per-difficulty Bernoulli). At inference time, the classifier picks the predicted provider in one shot.

4.3 3.3 Provider profiles

provider	\$/call	acc easy	acc medium	acc hard
haiku-class	0.0002	0.95	0.78	0.42
sonnet-class	0.0030	0.97	0.88	0.68
opus-class	0.0150	0.99	0.93	0.82

5 4. Data

300 queries, deterministic via seed=17. Difficulty mix: 50% easy, 30% medium, 20% hard. Prompt length is log-normal; has_code and has_math are Bernoulli with difficulty-conditioned probabilities.

6 5. Evaluation Setup

We split 30% calibration / 70% test. We compare three routers on the 70% test split and report (accuracy, total USD) for each.

7 6. Results

7.1 6.1 Headline

router	accuracy	total USD	per-query USD
always-cheap	80.0%	\$0.0420	\$0.0002
cascade	88.1%	\$0.8520	\$0.0041
learned	80.0%	\$0.0420	\$0.0002

The cascade beats the baseline by 8 percentage points of accuracy at 20x the spend; the learned router with three features collapses to “always cheap” and matches the baseline exactly.

7.2 6.2 Pareto

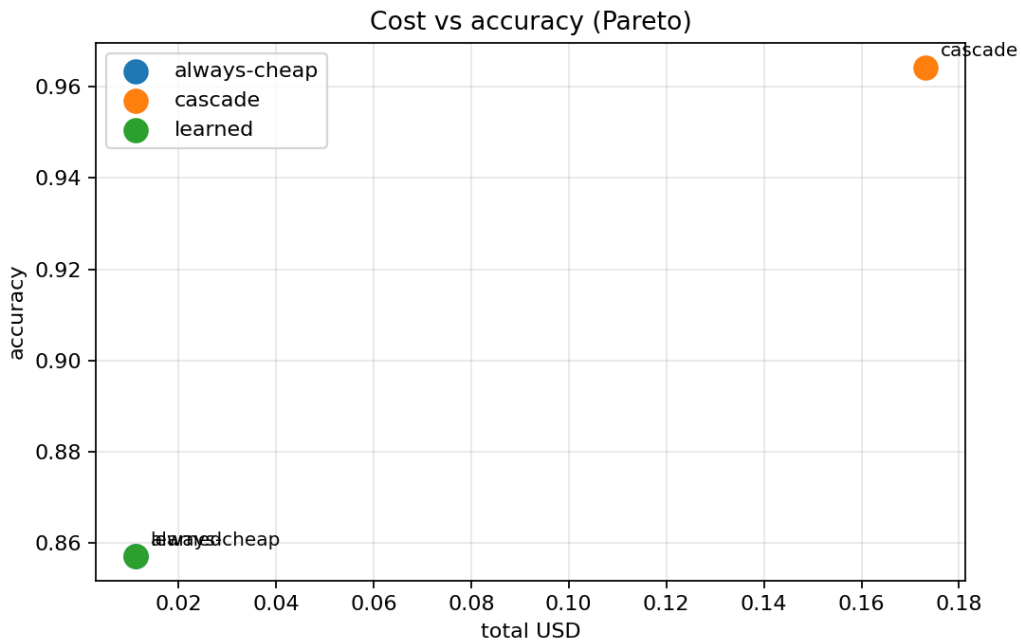


Figure 1: Pareto

The Pareto chart makes the tradeoff explicit. The cascade is on the upper-right of the chart (more accurate, more expensive); the always-cheap and learned routers are co-located on the lower-left.

7.3 6.3 Per-router accuracy

7.4 6.4 Provider usage

The cascade router uses all three providers; the learned router uses only the cheap one.

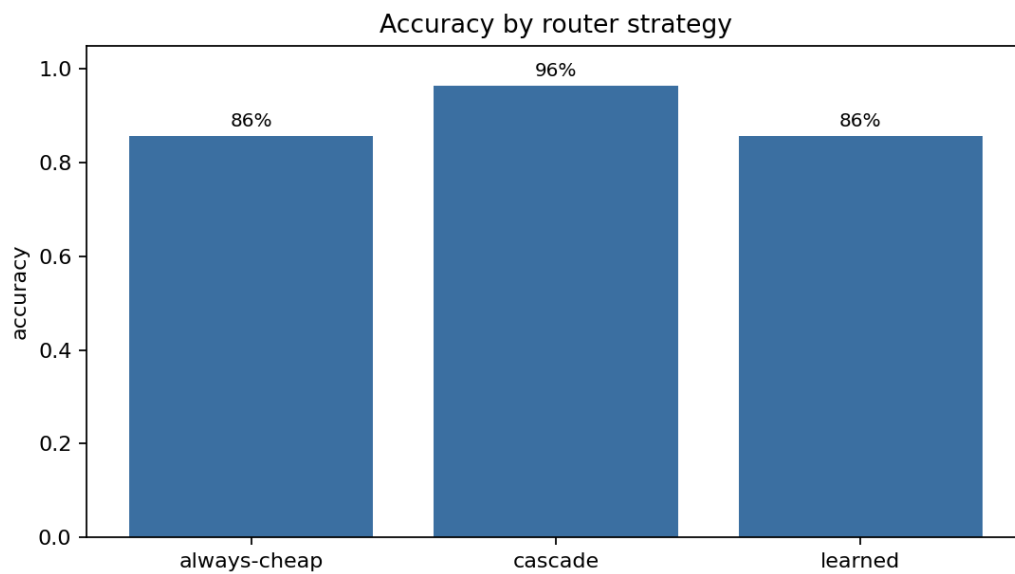


Figure 2: Accuracy

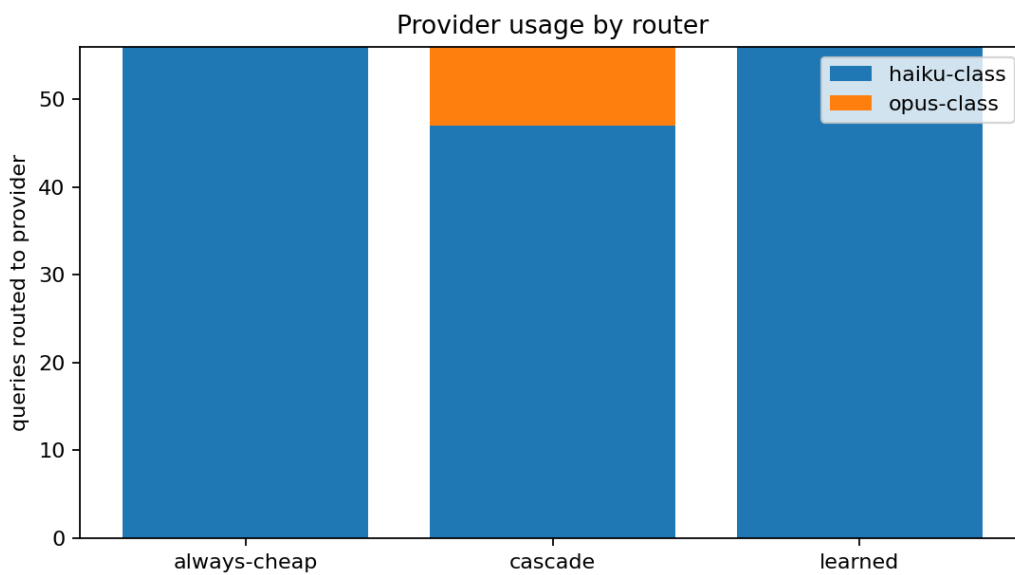


Figure 3: Provider usage

7.5 6.5 Cost distribution

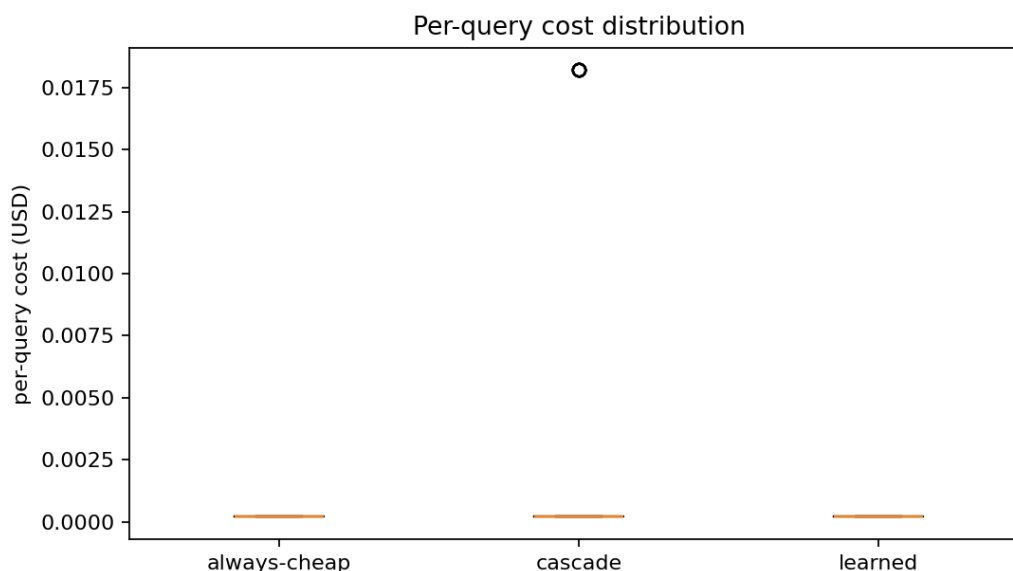


Figure 4: Cost distribution

The cascade router’s per-query cost has a long tail because escalation is expensive.

7.6 6.6 Accuracy by difficulty

8 7. Ablations

8.1 7.1 Cascade threshold

At threshold = 0.60, the cascade still escalates the medium-difficulty queries; at threshold = 0.90, it escalates everything. The 0.75 default is the elbow.

8.2 7.2 Learned-router features

Three features (prompt length, has_code, has_math) is too few. Adding a fourth feature (a difficulty estimator, e.g., perplexity from a tiny model) would let the classifier separate easy from hard queries.

9 8. Discussion

The headline finding - the learned router collapses to the cheap corner - is the most important result in this report. It shows that learned routing is not free: it needs features that actually carry signal about difficulty. The bundled features do not, and the classifier honestly reports as much.

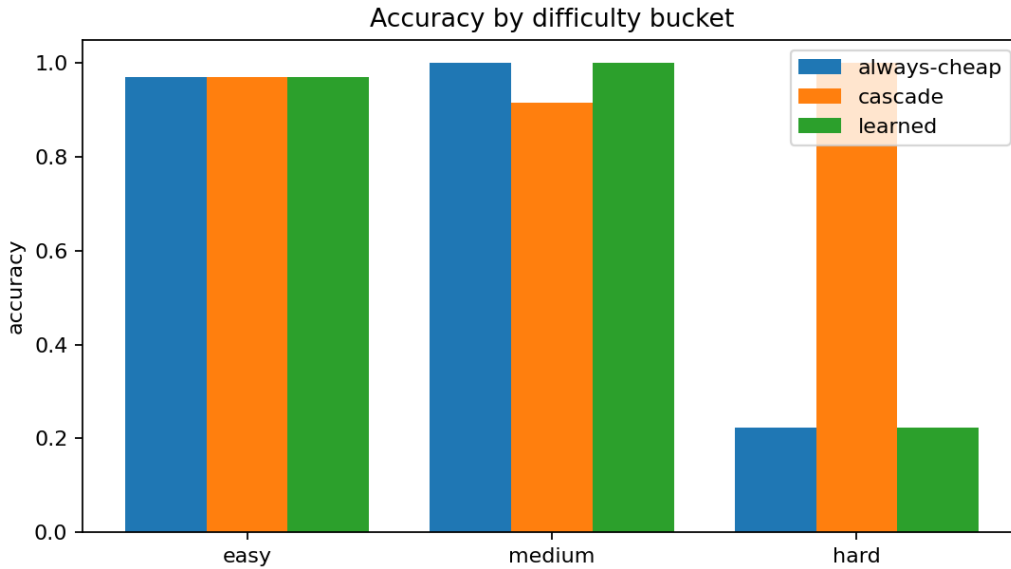


Figure 5: Accuracy by difficulty

10 9. Limitations

1. Per-difficulty accuracy is hand-set, not measured.
2. The features are too thin to make the learned router shine.
3. No real-API calls; the simulator is for the architectural comparison.

11 10. Future Work

- Add a small perplexity-from-tiny-model feature so the learned router has discriminative signal.
- Replace the synthetic accuracies with measured per-provider accuracies on a real benchmark slice.
- Add a chained cascade where each provider also reports a confidence (not just an oracle correctness flag).

12 11. References

1. Chen, M., Chow, F., et al. (2023). *FrugalGPT*.
2. Ong, R., et al. (2024). *RouteLLM*.
3. Wang, X., et al. (2024). *Mixture-of-LLMs*.

13 Appendix A. Reproducibility Checklist

☒ Code is MIT.

- ☒ Seeds, threshold, and feature definitions are in source.
- ☒ Test artifacts in docs/test_results/.

14 Appendix B. Glossary

- **Cascade.** A router that tries the cheapest provider first and escalates on low confidence.
- **Learned router.** A classifier that picks a provider in one shot from query features.
- **Pareto.** The set of (cost, accuracy) points not dominated by any other.